



Spectral clustering and query expansion using embeddings on the graph-based extension of the set-based information retrieval model

Nikitas-Rigas Kalogeropoulos^{*}, George Kontogiannis, Christos Makris

Computer Engineering and Informatics Department, University Campus, Rio, Patras, 26504, Achaia, Greece

ARTICLE INFO

Keywords:

Information retrieval
Information retrieval models
Set-based model
Graphical representation of textual data
Clustering
Spectral clustering
Graph embeddings

ABSTRACT

This paper presents a straightforward yet novel approach to enhance graph-based information retrieval models, by calibrating the relationships between node terms, leading to better evaluation metrics at the retrieval phase, and by reducing the total size of the graph. This is achieved by integrating spectral clustering, embedding-based graph pruning and term re-weighting. Spectral clustering assigns each term to a specific cluster, allowing us to propose two pruning methods: out-cluster and in-cluster pruning based on node similarities. In-cluster pruning refers to pruning edges between terms within the same cluster, while out-cluster pruning refers to edges that connect different clusters. Both methods utilize spectral embeddings to assess node similarities, resulting in more manageable clusters termed *concepts*. These *concepts* are likely to contain semantically similar terms, with each term's *concept* defined as the centroid of its cluster. We show that this graph pruning strategy significantly enhances the performance and effectiveness of the overall model, reducing, at the same time, its graph sparsity. Moreover, during the retrieval phase, the conceptually calibrated centroids are used to re-weight terms generated by user queries, and the precomputed embeddings enable efficient query expansion through a k-Nearest Neighbors (K-NN) approach, offering substantial enhancement with minimal additional time cost.

To the best of our knowledge, this is the first application of spectral clustering and embedding-based conceptualization to prune graph-based IR models. Our approach enhances both retrieval efficiency and performance while enabling effective query expansion with minimal additional computational overhead. Our proposed technique is applied across various graph-based information retrieval models, improving evaluation metrics and producing sparser graphs.

1. Introduction

Recent advancements in information retrieval have seen the integration of contextual embeddings, which consist of dense vectors, through various approaches, including vector-based methods and large language models (LLMs), to develop more robust and efficient models. However, due to latency concerns associated with these contextualized approaches, they are often combined with classical information retrieval models, such as BM25 (Robertson, Zaragoza, & Taylor, 2004), to strike a balance between efficiency and effectiveness. Typically, the initial sparse model serves as a first-stage ranker, generating a ranking that is subsequently refined by the contextualized model (Nogueira, Yang, Cho, & Lin, 2019; Wang et al., 2024). While independent “dense” models that utilize BERT-derived embeddings for ranking, such as those by Khattab and Zaharia (2020) and Sanh, Debut, Chaumond, and Wolf (2019), have demonstrated strong performance, their computational demands remain a challenge. In this landscape, semantic embeddings

have emerged as a means to further enrich models' document representations by incorporating deeper contextual understanding. However, beyond these approaches, there is a less explored yet promising domain of graph-based models, which we aim to expand upon in this work, building on our previous research (Kalogeropoulos, Doukas, Makris, & Kanavos, 2020; Kalogeropoulos, Skamnelos, Makris, et al., 2024). In graph-based models, documents or terms are represented as nodes in a graph (Sonawane & Kulkarni, 2014). These nodes can represent either individual documents or specific terms extracted from the documents and their associated relationships. We focus on the latter approach, where terms are represented as nodes. A common challenge in these types of models is the tendency for the graph to grow rapidly in size, leading to dense representations. This can significantly increase the computational cost of even simple graph operations, such as finding the shortest path between nodes. In some cases, the density of the graph

^{*} Corresponding author.

E-mail addresses: kalogeropo@ceid.upatras.gr (N.-R. Kalogeropoulos), g.kontogiannis@ac.upatras.gr (G. Kontogiannis), makri@ceid.upatras.gr (C. Makris).

<https://doi.org/10.1016/j.eswa.2024.125771>

Received 23 August 2024; Received in revised form 27 October 2024; Accepted 10 November 2024

Available online 19 November 2024

0957-4174/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

can also negatively impact the quality of the relationships between terms, resulting in decreased retrieval performance.

Our contribution. The primary objective of this paper is to introduce a novel extension of the Graph-Based Set-Based model, termed the Graphical Set-Based model (GSB) (Kalogeropoulos et al., 2020, 2024), designed to address these challenges in graph-based IR models. This method, however, is versatile and can be applied to any IR model that represents its corpus as a graph, such as those proposed by Rousseau and Vazirgiannis (2013, 2015). Our approach integrates the concept-based representation, a key feature of the classical information retrieval principle, Latent Semantic Indexing (LSI) (Deerwester, Dumais, Furnas, Landauer, & Harshman, 1990), to improve the expressiveness of the graph structure. By doing so, the new ranking model aims to enhance performance through a refined termset weighting scheme during the retrieval phase, utilizing Spectral Clustering and Embedding-based Conceptualization. Our focus extends beyond improving precision as a performance metric; we also aim to sparsify the graph without sacrificing information quality. To accomplish this, we explore clustering-based pruning techniques alongside the conceptualization method through spectral embedding. Moreover, the proposed approach generates embeddings that offer flexibility and can be adapted to various tasks. As a practical use case, we will demonstrate the use of these embeddings for query expansion to improve retrieval outcomes, with minimal additional computational cost.

The rest of this paper is organized as follows: Section 2 outlines the related work. Section 3 provides a theoretical overview of key methods and models relevant to this study, focusing on the extension of the set-based model with graph structures, including the graph construction process and an introduction to spectral embedding techniques that are integral to our approach. In Section 4, we present our contributions to the ranking task by introducing the proposed models and discussing their applications. Following that, Section 5 details the results obtained through the proposed methods. Finally, Section 6 addresses our conclusions, outlines the limitations of the study, and suggests directions for future research.

2. Related work

The area of Information Retrieval (IR) focuses on meeting users' informational needs, which are expressed as queries. An information retrieval system processes these queries and returns a ranked list of items that best match the user's requirements. Although the foundations of IR are built on the theory of information, nowadays, there is a tendency to combine classical methodologies with machine learning techniques. One such machine learning concept that has proven particularly valuable in the context of IR is the idea of embeddings. In Section 2.1 we present the fundamental techniques used to transform the plain and graph-based text into rich and meaningful vector representation (embeddings), while in Section 2.2 we present the state-of-the-art (SOTA) regarding graph-based IR models.

2.1. Embeddings

Embeddings are powerful and versatile tools that, in a broad sense, transform data from its original representation, such as words, nodes, or other objects, into a lower-dimensional space while preserving essential properties or relationships inherent in the data (Dimensionality Reduction). This transformation is beneficial when dealing with high-dimensional data, as it allows for the application of classical learning techniques such as classification and clustering, with increased efficiency and interpretability (Camacho-Collados & Pilehvar, 2018).

Word embeddings, such as Word2Vec (Mikolov, Chen, Corrado, & Dean, 2013) and GloVe (Pennington, Socher, & Manning, 2014), have become foundational algorithms in Natural Language Processing (NLP). These techniques have inspired the development of embeddings for more complex structures, such as graphs.

Moreover, contextualized embeddings, generated by models like BERT (Bidirectional Encoder Representations from Transformers) (Devlin, Chang, Lee, & Toutanova, 2018) alongside the work of Vaswani et al. (2017) known as the attention mechanism, have revolutionized NLP. Unlike static embeddings that assign a single representation to each word, contextualized embeddings capture a word's meaning based on its surrounding context within a sentence. This allows for a deeper understanding of language, improving performance in various NLP tasks such as translation, sentiment analysis, and question answering. These embeddings adjust dynamically based on surrounding words, providing richer, context-aware representations. Contextual embeddings have advanced information retrieval, leading to the development of dense retrieval methods. An example is the ColBERT model by Khattab and Zaharia (2020). This model leverages BERT embeddings to compute a similarity function between query and document terms using cosine similarity on their embeddings.

In parallel, the field of graph embeddings, has evolved to handle data that is naturally represented as graphs. Graphs are complex structures rich in intricate relationships and properties, however, often difficult to interpret. Graph embeddings aim to represent each node in a graph as a low-dimensional vector while preserving the graph's structural properties. The goal is to ensure that nodes that are close in the graph remain close in the vector space. Notable models in this area include Node2Vec (Grover & Leskovec, 2016) and techniques based on Spectral Embeddings, both of which have shown strong performance in capturing the essence of graph structures.

Moving beyond the individual nodes, it is also important to embed structural knowledge from the graph at a higher level. This process involves capturing the overall structure and relationships within the graph, allowing for a more comprehensive understanding of the data. Such embeddings can include information about clusters, subgraphs, or global properties, which are essential for tasks that require an understanding of the broader context within the graph. One effective model for this is the Graph2Vec (Narayanan et al., 2017), which generates embeddings by learning representations of entire graphs, similar to how document embeddings represent entire documents. The Graph2Vec model captures both local and global structural information, facilitating various graph-based machine-learning tasks.

The most common methods for generating node embeddings are based on matrix factorization techniques, random walks algorithms, and graph neural networks. Matrix factorization-based methods reduce the dimensionality of the adjacency matrix or a related graph matrix through factorization. A classic approach to this is the spectral decomposition of the Laplacian matrix, which forms the basis of spectral clustering. An alternative approach, derived from the (generalized) Singular Value Decomposition (SVD) of the adjacency matrix, is also closely related to the spectral approach (Belkin & Niyogi, 2003).

Techniques such as DeepWalk (Perozzi, Al-Rfou, & Skiena, 2014), Node2Vec (Grover & Leskovec, 2016), and LINE (Tang et al., 2015) perform random walks on the graph to preserve local structures and neighborhoods. These methods generate sequences of nodes in the manner of random walks and then apply methods similar to Word2Vec to learn the embeddings of the nodes. Alongside with such methods, Graph Neural Networks (GNNs) are a distinct class of deep learning models capable of learning node representations by utilizing both node features and graph topology. Notable examples include Graph Convolutional Networks (GCNs) (Kipf & Welling, 2016), Graph Attention Networks (GATs) (Veličković et al., 2017), and GraphSAGE (Hamilton, Ying, & Leskovec, 2017).

2.2. Graph-based information retrieval models

Graph-based approaches in IR have been explored due to their ability to model complex relationships between documents, terms, and queries. This section reviews state-of-the-art methods that have leveraged graph structures for improved retrieval performance.

PageRank (Page, Brin, Motwani, & Winograd, 1999) is one of the earliest and most influential graph-based models, originally developed for ranking web pages. It treats the web as a directed graph where each webpage is a node, and hyperlinks are the edges. The PageRank algorithm computes a global ranking of nodes based on their link structure, assuming that more important nodes receive more links from other important nodes. Although PageRank was not initially designed for traditional text retrieval, its principles have been applied in various ways to model relationships between documents and terms.

TextRank (Mihalcea & Tarau, 2004) adapts the principles of PageRank to extract key phrases and rank sentences in textual data. It represents a text as a graph where sentences (or words) are nodes, and edges indicate semantic or syntactic relationships. TextRank has been particularly effective in keyword extraction and summarization tasks. By representing sentences as nodes and establishing weighted edges based on content similarity, TextRank ranks the most important sentences, providing a concise representation of the text's main ideas.

Heterogeneous Information Networks (HINs) (Sun, Han, Yan, Yu, & Wu, 2011) have been explored for their ability to model diverse types of objects and relationships in a single graph structure. HINs have been particularly useful in complex IR settings, where different entities (e.g., authors, papers, venues) interact. The use of meta-path-based similarity measures allows for effective retrieval by capturing the semantics of different relationships in the network.

Graph-based neural ranking models, such as the work of Dai and Callan (2019), leverage both graph structures and deep learning. These models use graphs to represent relationships between queries and documents and utilize neural architectures to learn the ranking function directly from the graph. This approach has demonstrated improvements in text understanding and ranking accuracy by capturing contextual nuances between queries and documents.

Our novelty. Building on the foundational studies and analyses from our earlier work, this research introduces a novel use of Spectral Clustering (SC) and Embedding-based Conceptualization to address challenges such as excessive graph size and poor term relationship quality. Unlike previous studies that predominantly focus on generating rich embeddings through deep neural networks, our approach targets the less-explored domain of spectral embeddings. We utilize clusters created by SC, referred to as *concepts*, to calibrate the termset weighting scheme during the retrieval phase. Simultaneously, the embeddings generated for the node terms are leveraged to introduce two novel pruning methods, significantly reducing the graph size without compromising information quality. In our proposed approach we will not elaborate on the clustering optimization issue, as in the work of Borlea, Precup, and Borlea (2022), Borlea, Precup, Dragan, and Borlea (2017) and Gálvez, Cuevas, Becerra, et al. (2020). However, we utilize the number of cluster as parameter during the cluster generation, which later will be used as indicators for pruning or in the conceptualization process of a collection-wide graph. Furthermore, we enhance retrieval performance through query expansion using pre-computed spectral embeddings, demonstrating their effectiveness in capturing conceptualized meaning. This approach not only broadens the application of machine learning in IR models but also positions graph-based IR models as competitive alternatives to trending LLM-based IR models.

3. Preliminaries and methods

This section provides an overview of the set-based model's extension implementing graphs, both with and without windowing techniques, and a brief overview of the Spectral clustering. In later sections, these components play a crucial role in enhancing the model's performance and adaptability.

3.1. Spectral embedding–spectral clustering

Spectral Clustering (von Luxburg, 2007) is a technique used for clustering data by leveraging the properties of eigenvalues (the spectrum) of a similarity matrix. The process begins by constructing a similarity matrix that measures similar data point pairs. From this matrix, a Laplacian matrix is derived, capturing the underlying structure of the data as a graph. Spectral Clustering then involves computing the eigenvalues and eigenvectors of the Laplacian matrix to obtain a lower-dimensional representation of the data. This reduced representation is then input for a clustering algorithm, such as k-means. Spectral Clustering is particularly effective for identifying clusters in data that is not linearly separable, making it useful for uncovering complex cluster structures.

Spectral Embedding (Bonald, 2019), on the other hand, refer to the representation of data points in a lower-dimensional space using eigenvectors obtained from spectral methods. These embeddings are derived from the spectral decomposition of matrices associated with the data, such as the adjacency matrix of a graph or the Laplacian matrix. By transforming high-dimensional data into this lower-dimensional space, spectral embeddings aim to preserve the intrinsic geometric structure of the data. This approach is beneficial for tasks such as clustering, visualization, and as features for machine learning models, as it captures the essential relationships and connectivity within the data. Both Spectral Clustering and Spectral Embeddings are powerful techniques for analyzing complex data structures and uncovering patterns that might not be apparent in the original high-dimensional space.

3.2. Graph based extension of set-based model

The approach based on graphs (Graphical Set-Based Model, GSB) by Kalogeropoulos et al. (2020), extends the Set-based model of Possas, Ziviani, Meira, and Ribeiro-Neto (2002). In this approach, term indexing is achieved through graphs, while the representation of texts and queries remains similar to the previous model. Moreover, GSB follows a different path regarding weight calculation. By analyzing the graphs of texts into subgraphs known as cores (K-core Decomposition) by the work of Seidman (1983), it introduces the concept of node importance, considering them as keywords. Each text is represented as an undirected weighted graph.

3.2.1. Rational path graph

During graph creation, GSB assumes that each term is connected to the other terms of the text. Thus, each term of a specific text is represented as a unique node, and the relationship between two terms is represented as an edge with the corresponding nodes as the start and end. Each edge is assigned a weight, symbolized as W_{out} , and each node has an internal edge with the same node as the start and end, with a weight symbolized as W_{in} . They offer two alternatives, based on the document input, for the graph construction process known as the Rational Path Graph. The first consists of the two following theorems:

Theorem 3.1. For a node N_i belonging to a text D_j , with term frequency tf_i , the weight of the internal edge (N_i, N_i) is calculated as:

$$W_{in} = \frac{tf_i \times (tf_i + 1)}{2} \quad (1)$$

Theorem 3.2. For two nodes N_i, N_j belonging to a text D_j , with term frequencies tf_i, tf_j respectively, the weight of the external edge (N_i, N_j) is calculated as:

$$W_{out_{i,j}} = tf_i \times tf_j, \text{ when } N_i \neq N_j \quad (2)$$

The previously described method is applicable when documents are provided as inverted lists. However, in scenarios involving raw text data, an alternative algorithm is preferred, which represents the text as a stack. In this approach, for each term in the stack, upon removal, an edge is established with every other word remaining in the stack. If the edge does not already exist, it is created, or the weight of the existing edge is incremented by one. This process results in the generation of the same graph structure as the first method.

3.2.2. Union graph

After creating individual graphs for each text and identifying significant nodes, a comprehensive union-wide graph U is composed, including nodes and edges from all texts. Edge weights are calculated as the total sum of the corresponding weights from each graph (Sonawane & Kulkarni, 2014).

The process of updating the overall graph U is as follows: Initially, U is an empty graph. In each iteration of the algorithm, a text graph G is added. Each term t of G is checked for significance, and a value h is defined. For non-significant terms, $h = 1$. If the term exists in U , then the internal weight is added to the corresponding weight of G , multiplied by the value h . Then, the values of all external edges for term t are updated. The weight of each node k in the final graph U is calculated as:

$$NW_k = \log \left(1 + a \times \frac{W_{outk}}{(W_{ink} + 1) \times (n_{gk} + 1)} \right) \times \log \left(1 + b \times \frac{1}{n_{gk} + 1} \right) \quad (3)$$

where W_{outk} and W_{ink} are the total sums of external and internal edges of the node k , n_{gk} is the number of neighboring nodes, and a, b are parameters intended to demonstrate the improvement effect of GSB compared to the Set-based model.

3.2.3. Retrieval

In the GSB, the information retrieval process aligns with the Set-based Model. The process begins with the creation of term sets using the Apriori algorithm. Subsequently, new sets are formed from two sets that differ by only one term.

For each set S_i , a metric TNS_i is calculated, representing the product of the graph weights for each term in the set:

$$TNS_i = \prod_{k \in S_i} NW_k \quad (4)$$

This metric is then integrated into the formula for calculating the weight of a term set S_i in a document D_j , where Sf_{ij} represents term frequency (TF) and Nd_{Si} represents inverse document frequency (IDF):

$$WS_{ij} = (1 + \log Sf_{ij}) \times \log(1 + Nd_{Si}) \times tnw_{Si} \quad (5)$$

The formula for the query weight termset is calculated with the following equation and practically is the same as the inverse document frequency of the set.

$$WS_{iq} = \log(1 + Nd_{Si}) \quad (6)$$

Therefore, the document and query vectors are created.

$$\vec{d}_j = (WS_{1j}, WS_{2j}, \dots, WS_{2n_j}) \quad (7)$$

$$\vec{Q} = (WS_{1q}, WS_{2q}, \dots, WS_{2n_q}) \quad (8)$$

The size of the vectors cannot exceed $2n^2$, where n is the number of words in the query. To calculate the similarity between documents and the query, cosine similarity is used:

$$\text{sim}(\vec{Q}, \vec{d}_j) = \frac{\vec{d}_j \cdot \vec{Q}}{\|\vec{d}_j\| \times \|\vec{Q}\|} \quad (9)$$

The results of this similarity will form the scoring function of the model, which by sorting will offer the final ranking of the model.

3.2.4. Fine tuning the rational path graph

The GSB constructs complete text graphs as discussed in Section 3.2.1, assuming that every word is equally related to the others (Kalogeropoulos et al., 2020). This simplistic approach in its core seems linguistically flawed, especially in collections that contain large documents. It creates issues in identifying significant nodes. A naive solution to such a problem is to remove certain edges after the document graphs have been created, thereby differentiating the relationship between words based on the content of sentences and paragraphs. However, that approach depending on the chosen edge weight threshold, might result in information loss on the union graph.

In the recent work of Kalogeropoulos et al. (2024), the notion of document segmentation is employed on the graph-based extension of the set-based model (GSB) that yields better results on the ranking task. The text is divided into segments called windows, which may refer to sentences or paragraphs depending on the approach.

More specifically, a document segment may correspond to a sentence, with its size being determined either as a percentage of the document's total length or as a small single-digit number. Rousseau and Vazirgiannis (2013) in their work, suggested a range of 6 to 30, though they observed that, in their sliding window approach, this range had minimal effect on the model's precision. Conversely, the model proposed by Kalogeropoulos et al. (2024) shows that window size significantly impacts model precision, as the sentence window is non-overlapping, advancing by a number of terms equal to the chosen size.

They divide the document as outlined and then apply the rational path graph creation process to each segment. The resulting graph consists of cliques, which represent the sentences. These cliques are typically linked by bridge edges, which are common edges shared among the cliques. After this, the model proceeds as described in the preliminary section.

In the extreme case where no bridge edge exists, the document graph will consist of two separate subgraphs. The model can handle this situation without issues and will update the union graph accordingly. From a linguistic perspective, one could hypothesize that the document covers multiple thematic topics, or that there may be a synonym issue. A synonym issue arises when different words with similar meanings are used in various segments, causing the model to treat them as unrelated, even though they share semantic connections. However, document graphs that are more distinct and indicative of the content are generated compared to the simple GSB model.

Exploring the Windows approach in more detail, three alternatives were proposed based on the natural language flow. Initially, they hypothesized that the relationship among terms exists only within a sentence, implementing a constant-sized window defined by the user. However, this method had a notable drawback: the window size slightly depended on the collection. Even in the worst cases, the results were equal to or better than the complete model, as larger window sizes approximated the complete model, and a cut-off of four terms was applied for smaller values since a proper sentence rarely contains fewer than three words. To address this drawback, they introduced a dynamic window size based on a percentage of the document's total length, creating a percentage-sized window.

Finally, they explored the hypothesis that while terms within a sentence have a strong relationship, terms within a paragraph have a weaker one. Therefore, each edge weight is calculated as a linear combination of the sentence and paragraph edge weights, multiplied by importance variables α and β , respectively.

Consider the document "T1 T2 T3 T2 T4 T5 T1 T3 T5 T4 T1 T2 T4 T6". Additionally, let us assume that the window size is set to 3 as shown on Table 1.

Table 1
Windowed example.

Doc terms	T1	T2	T3	T2	T4	T5	T1	T3	T5	T4	T1	T2	T4	T6
Window		1			2			3			4			5

Table 2
Dynamic windowed example.

Doc terms	T1	T2	T3	T2	T4	T5	T1	T3	T5	T4	T1	T2	T4	T6
Window			1				2				3			4

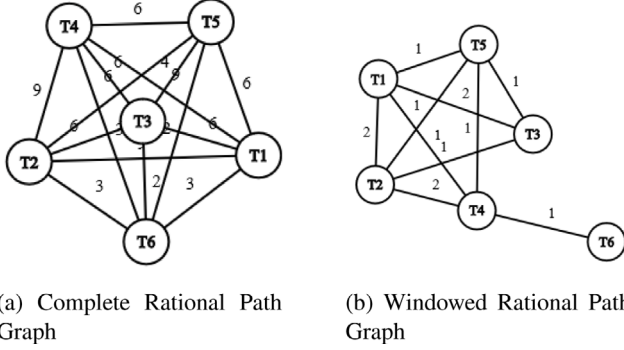


Fig. 1. Comparison of complete and windowed rational path graphs.

On the complete case discussed in Section 3.2.1 the graph created by the algorithm from the document in the above example is depicted in Fig. 1(a) and the Fig. 1(b) shows the document graph implementing windows.

Furthermore, on the dynamic windowed case the user, as mentioned before, can define a percentage of the document size in order to differentiate the segments of each document. If the percentage in the above example is equal to 35% the window size will be 4 terms and the generated segments are shown in Table 2

4. Proposed methods

This section aims to address the completeness of rational path graphs and to refine and model the relationships among terms in a document more appropriately. Initially, the edge pruning methods proposed by Zhou, Mahler, and Toivonen (2012) were considered. Still, due to concerns about potential information loss, the windowed approach seemed more reasonable since it appeared to be a more linguistically correct option.

First, we apply clustering-based pruning methods for every graph-based extension of the set-based model (whether complete or windowed). This will result in more cohesive and smaller union graphs, thereby aiming to generate more appropriate node weights. In a subsequent phase, we introduce the notion of the *concept* as an outcome of the clustering process. We hypothesize that each cluster contains semantically and syntactically similar terms, thereby generating a *concept* for each cluster.

4.1. Pruned graphical set-based

The proposed method continues after creating the union graph of the simple graph-based extension of the set-based model as mentioned in Sections 3.2 and 3.2.4. The collection-wide union graph is transformed into the corresponding weighted adjacency matrix. After that, spectral clustering is applied, producing both the cluster labels for each node and a vector representation of the nodes in an embedded space as shown in algorithm 1

Next, the graph is pruned using a combination of two distinct clustering methods, each deeply influenced by the underlying cluster

Algorithm 1: Cluster Graph using Spectral Clustering

Input: *graph*: Input graph, *collection*: Node collection, *n_clstrs*: Number of clusters
Output: *graph*: Graph with assigned cluster labels, *embeddings*: Spectral embeddings

- 1 *adj_matrix* $\leftarrow$ Convert *graph* into an adjacency matrix representation;
- 2 Create Spectral Embedding with *n_components* = *n_clstrs* and affinity as 'precomputed';
- 3 *embeddings* $\leftarrow$ Compute embeddings using Spectral Embedding on *adj_matrix*;
- 4 Initialize KMeans clustering with *n_clusters* = *n_clstrs*;
- 5 *labels* $\leftarrow$ Compute labels by applying KMeans clustering to *embeddings*;
- 6 **for each edge** (*u*, *v*) **in** *graph* **do**
- 7 *c* $\leftarrow$ Collection index corresponding to *u*;
- 8 *w* $\leftarrow$ Collection index corresponding to *v*;
- 9 Assign cluster label *labels*[*c*] to node *u*;
- 10 Assign cluster label *labels*[*w*] to node *v*;
- 11 **return** *graph*, *embeddings*;

structure in the data. We aim to capture both inter-cluster and intra-cluster relationships. Inter-cluster relationships specifically target the edges connecting different clusters or groups, highlighting their differences. Conversely, intra-cluster clustering focuses on the relationships within a single cluster, seeking to uncover subtle patterns and connections among similar elements throughout the union graph. Therefore we consider different thresholds to capture the inter- or intra-cluster connection. Moreover, we employ two different pruning methods: one based on edge weights (Eq. (10)) and the other on the similarity of node embeddings between connected nodes (Eq. (12)).

To decide whether the edge between nodes *u* and *v* will stay in the graph, we use a pruning function termed in the sequel as P(), which when the value is 1 removes the edge (*u*, *v*). This occurs on every case presented in this section.

On the following schema the function is based on the edge weight. For a graph $G = (V, E)$ with vertices *V* and edges *E*, the edge pruning function P_{edge} is defined as:

$$P_{\text{edge}}(u, v) = \begin{cases} 1 & \text{if } w(u, v) \leq \theta \text{ and } C_u \neq C_v \\ 1 & \text{if } w(u, v) \leq 2\theta \text{ and } C_u = C_v \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

where $w(u, v)$ denotes the weight of the edge between *u* and *v*, and:

$$\theta_u = \frac{\sum_{v \in N(u)} w(u, v)}{|N(u)|} \quad (11)$$

where $N(u)$ are the neighboring vertices of *u*.

Given a graph $G = (V, E)$ and the node embeddings $\mathcal{E}_u$ and $\mathcal{E}_v$, the pruning function based on cosine similarity, P_{sim} is defined as:

$$P_{\text{sim}}(u, v) = \begin{cases} 1 & \text{if } \text{sim}(\mathcal{E}_u, \mathcal{E}_v) \leq \theta \text{ and } C_u \neq C_v \\ 1 & \text{if } \text{sim}(\mathcal{E}_u, \mathcal{E}_v) \leq 2|\theta| \text{ and } C_u = C_v \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

where $\text{sim}(\mathcal{E}_u, \mathcal{E}_v)$ is the cosine similarity of the node embeddings u and v , and C_u and C_v represent the clusters to which the nodes u and v belong, respectively. The value of θ must lie in the range $[-1, 1]$ to be consistent with the range of possible values for cosine similarity, ensuring that pruning is coherent with the geometric interpretation of the embeddings.

Algorithm 2: Prune Graph Based on Threshold Condition

Input: *graph*: Input graph, *condition*: Pruning condition
Output: *graph*: Pruned graph

```

1 Extract cond,  $\theta$  from condition;
2 for each edge  $(u, v)$  in graph do
3   Set  $C_u$  and  $C_v$  as the clusters of nodes  $u$  and  $v$ , respectively;
4   if cond = 'edge' then
5      $\text{weight} \leftarrow$  weight of edge  $(u, v)$ ;
6      $\text{flag} \leftarrow \text{weight} \leq \theta$ ;
7   else if cond = 'sim' then
8     Compute similarity between embeddings of  $u$  and  $v$  as
       similarity;
9      $\text{flag} \leftarrow \text{similarity} \leq \theta$ ;
10  if  $(C_u \neq C_v \text{ and } \text{flag})$  or  $(C_u = C_v \text{ and } ((\text{cond} = \text{'sim'} \text{ and } \text{similarity} \leq 2 \cdot |\theta|) \text{ or } (\text{cond} = \text{'edge'} \text{ and } \text{weight} \leq 2 \cdot \theta)))$ 
    then
11    Remove edge  $(u, v)$  from graph;
12 return graph;
```

After the sparsification process is completed a new union graph emerges and thereafter the indexing of the terms is continued by calculating the new NW_k based on Eq. (3). Finally, the retrieval process can be concluded as described in Section 3.2.3.

4.2. Conceptualized graphical set-based

The conceptualized version of the graph-based extension is inspired by the pruned graph-based extension of the set-based model (PGSB) or the windowed version of the model (PGSBW) and the Latent Semantic Indexing (Deerwester et al., 1990). We can hypothesize that each cluster of terms contains semantic similar concepts. Therefore, we can represent each term t of the cluster C_k with the centroid concept. For that reason we define the new concept weight as C_{NW_k} :

$$C_{NW_k} = \frac{\sum_{t \in C_k} NW_t}{|C_k|} \quad (13)$$

where NW_t is the weight for term t computed using Eq. (3) and $|C_k|$ denotes the number of elements in cluster C_k . This approach transitions us into a conceptual space, reducing the number of terms that need to be indexed.

However, before introducing the models within the conceptualized framework (CGSB), it is essential to note that they are all closely tied to the embeddings. This means that the effectiveness of these models, regardless of the strategy employed, depends heavily on the quality of the embeddings. Initially, the dimensions of an embedding derived from spectral clustering align with the selected size of the created clusters. Although, this does not consist of a size limit, the selected dimension should be less than or equal to the anticipated number of clusters in the data. This is because the spectral clustering method leverages the eigenvectors corresponding to the smallest eigenvalues of the Laplacian matrix, generating lower-dimensional embeddings that maintain the original structure. In our approach the embeddings size is determined by the eigengap of two consecutive eigenvalues. The rationale behind this method is to choose the dimension where the largest eigenvalue appears, representing a significant reduction in the variance explained by the additional dimensions. Mathematically, this

can be expressed by defining the number of dimensions (columns) to be retained from the total embeddings k .

The philosophy by which the simple extension assigns weight to a term node is entirely dependent on the number of its neighbors. That is, the greater the number of neighbors a term node has, the less weight (importance) it is assigned. In the union graph, therefore, there are term nodes with many external edges connected to term nodes belonging to different clusters. Thus, for CNW_k to represent as accurately as possible the importance of each term, it should not be influenced by the edge weights between clusters. Hence, in this specific model, pruning the graph is done as follows:

Given the union graph $G = (V, E)$ with C_u, C_v representing the clusters to which nodes u and v belong, respectively, the pruning function is defined as:

$$P_{\text{cluster}}(u, v) = \begin{cases} 1 & \text{if } C_u \neq C_v \\ 0 & \text{otherwise} \end{cases} \quad (14)$$

The CGSB based on clusters, while highly effective as we will see in the experimental results and discussion chapter, employs a very aggressive pruning strategy that results in a disconnected subgraph with n clusters. This is undesirable because the models proposed thus far rely on embeddings, whose quality is critically important. Laplacian embeddings face challenges with disconnected graphs due to the presence of many zero eigenvalues associated with disconnected components. This leads to spectrum compression, often causing embeddings to collapse along the diagonal between disconnected components, regardless of their dimensionality. Consequently, significant information about the graph's structure is lost, effectively eliminating relative information, thereby hindering clustering quality and parameter selection (Belkin & Niyogi, 2003).

For this reason, a combined pruning strategy that leverages the graph's quality metrics is necessary once more. The models proposed in this section adopt the pruning strategies outlined in Section 4.1 on the Eqs. (10), (12), and (14).

The main difference is the metric used to compute the node weights, either implementing the cosine similarity of their embeddings or the average weight of their edges. On every case the node weights are calculated based on Eq. (13) and ensure that each node's importance is reflected in the pruning process. Moreover, this combination will incorporate inter- and intra-cluster information on the union graph, as discussed on the 4.1 section.

4.3. Query expansion implementing spectral clustering

A key aspect of our approach is that spectral clustering generates embeddings, which can later be applied in various supervised or unsupervised machine learning tasks. One of the most common applications of these techniques is text classification, where the resulting vectors can capture semantic, structural, or even contextual information (Selva Birunda & Kanniga Devi, 2021).

Moreover, models can utilize semantic embeddings to enhance query text by incorporating collection terms, thereby improving the retrieval process. For example, Diaz, Mitra, and Craswell (2016) demonstrated that query expansion using multiple models can improve ranking performance, even when the embeddings are generated on a local or global scope within the collection. Similarly, Roy, Paul, Mitra, and Garain (2016) achieved comparable results by employing a deep learning classifier for query expansion. By creating the embedded space where embeddings reside, query expansion naturally emerges as a by-product that can significantly enhance semantic relationship in the query.

Therefore, we can effectively leverage the generated embeddings to perform query expansion using k-Nearest Neighbors (k-NN) method, which, offered substantial improvements with negligible additional time cost. This process is described in Algorithm 3, which outlines the steps needed for the supplementary expansion of the query with the

query terms that are closest in the embedded space and the best nearest neighbors, allowing the query to be analyzed, increasing its volume and offering better results regarding term relationships, as we will discuss in the later section.

In this approach, we include terms present in the query but absent in the collection index (as they cannot be retrieved) into the expanded query. We focus only on the common terms between the inverted index and the query. Consequently, the k-Nearest Neighbors are computed using the same embedding model, which requires minimal time. These additional query terms are then incorporated into the newly expanded query.

Algorithm 3: Query Expansion using Pre-calculated Embeddings

Input: query Q , number of neighbors k
Output: Expanded terms $\mathcal{E}$

```

1  $\mathcal{E} \leftarrow \{term \mid term \in query, term \notin inverted\_index\}$ 
  // Expansion terms;
2  $Q \leftarrow \{query\_embeddings\}$  // Query embeddings;
3  $q \leftarrow \text{centroid of } Q$  // Query centroid in embedding space;
4 Initialize K-NN model with embeddings using cosine metric;
5 Fit K-NN model on embeddings;
6 Find  $k$  nearest neighbors to  $q$ , store as  $N$ ;
7 foreach  $c_i \in N$  do
8    $C \leftarrow \{keys \text{ in } inverted\_index\}$  // Collection terms;
9   foreach  $c_j \in C$  do
10    if  $c_j \notin query$  then
11      if  $C[index] \notin query$  then
12         $\mathcal{E} \leftarrow \mathcal{E} \cup \{C[index]\}$ ;
13      end
14    end
15  end
16 end
17 return  $\mathcal{E}$ ;

```

The reflection of the terms is scattered using the k-Nearest Neighbors method. The k-Nearest Neighbors to the query centroid are identified. These neighbors, representing the closest terms in the embedded space, are incorporated into the newly expanded query. This method segregates and adds the additional terms to the query, resulting in improvements and yielding better precision results. This approach is even though simple, offers a very good extension of the algorithmic process.

Finally, it is important to note that this technique can be applied to all information retrieval models using any known word embedding method. However, we use the spectral solution because it is implemented in the union graph pruning process, thereby reducing complexity.

All methods proposed in Sections 4.1–4.3 can be implemented on complete document graphs or on document graphs created by using text segments as windows. In the retrieval process the set based model is implemented as described in Section 3.2. To be more specific, after computing the weights for each term-node k , the total index is constructed and the retrieval and ranking phase follows.

In contrast to Eq. (4) of GSB and PGSB(w), the (TN_{S_i}) of CGSB is defined as:

$$TN_{S_i} = \prod_{k \in S_i} CN_{W_k} \quad (15)$$

In the retrieval phase, these semantically multiplied concepts are used to rank the terms generated by the user's queries, based on Eq. (15).

5. Tests and evaluation

In this section, we aim to evaluate the different information retrieval proposed techniques based on the ranking they produce and not on the generated clusterings. Thus, we will employ precision based metrics for our evaluation. We will compare the Mean Average Precision (MAP) of the proposed models on a document collection with those described in the preliminaries section (see Section 3), as well as with several state-of-the-art models, including BM25 (Robertson & Sparck Jones, 1977) and colBERT (Khattab & Zaharia, 2020). We will elaborate on our findings and subsequently apply a comprehensive metric to evaluate the models' performance, taking into account additional parameters and the precision of each model.

5.1. Collections and experimental setup

We will use the Cystic Fibrosis (Shaw et al., 1991) document collection that includes 1239 abstracts related to the disease, along with a hundred (100) queries paired with expert-generated relevant document lists that can be used to evaluate ranking performance. The source code and data can be found in the [link](#).

Table 3 presents the models and important parameters used in our study. Both the PGSB and CGSB models treat the document rational path graph as a complete graph, while the last four models incorporate text segments as windows. The $PGSBW_{const}$ and $CGSBW_{const}$ models use a fixed window size of seven terms, whereas the $PGSBW_{per}$ and $CGSBW_{per}$ models implement a dynamic window size based on the total length of each document. Therefore, the size is calculated as a percent of the document size, which considered percentages are shown on the third column of the table. The columns *cluster Sizes*, *Similarities*, *Edge Coefficients* and *Window Percentages* of Table 3 are the grid in which we will search for the optimal model.

The comparison is conducted across all the proposed models, but these comparisons will not rely solely on mean average precision (MAP), as doing so would be overly simplistic. All the new models aim to address document retrieval from a conceptual standpoint. This approach introduces the risk that some models might achieve very high accuracy on certain queries while performing poorly on others, resulting in an overall high MAP. In an information retrieval model, all queries must be performed consistently. To prevent misleading evaluations of the models, a new evaluation metric is introduced in Eq. (16). This metric employs the number of queries each model is over-performs against the others, as well as the union graph size that the model needed to produce such results.

$$eval_i = \alpha \left(\frac{\sum_{j \neq i} positives_{ij}}{(n-1) \cdot 100} \right) + \beta \left(\frac{\sum_{k=1}^Q P_k}{Q} \right) + \gamma \left(\frac{E_p}{E} \cdot 100 \right) \quad (16)$$

where:

- α, β, γ : Coefficients for weighting the different components of the metric that $\alpha + \beta + \gamma = 1$.
- $positives_{ij}$: The number of questions where model i outperforms model j .
- n : Total number of models being compared.
- Q : Total number of questions.
- P_k : Precision for question k .
- E : Total number of edges in the original graph.
- E_p : Number of edges pruned from the graph.

Given the substantial amount of information that must be processed in current information retrieval models, it would be simplistic to evaluate a model solely based on its ranking performance. Equally or almost equally, precise models should be evaluated by their respective union graph, needed to capture the necessary information. To address this, we implemented the heuristic evaluation method described in (16), which accounts for both the size of the union graph, thereby assessing

Table 3

Cluster sizes, similarities, edge coefficients, and window percentages for different models.

Model	Cluster sizes	Similarities (%)	Edge coefficients	Window percentages
PGSB	30, 50, 70, 90, 110, 130, 150, 170	0.1, 0.3, 0.5, 0.7	1.1, 1.2, 1.3, 1.4	
CGSB	30, 50, 70, 90, 110, 130, 150, 170	0.1, 0.3, 0.5, 0.7	1.1, 1.2, 1.3, 1.4	
PGSBW _{const}	30, 50, 70, 90, 110, 130, 150, 170	0.1, 0.3, 0.5, 0.7	1.1, 1.2, 1.3, 1.4	
CGSBW _{const}	30, 50, 70, 90, 110, 130, 150, 170	0.1, 0.3, 0.5, 0.7	1.1, 1.2, 1.3, 1.4	
PGSBW _{per}	110, 130, 150, 170	0.5, 0.7	1.3, 1.4	0.1, 0.3, 0.5, 0.7
CGSBW _{per}	110, 130, 150, 170	0.5, 0.7	1.3, 1.4	0.1, 0.3, 0.5, 0.7

the model's space complexity, and the Mean Average Precision, which reflects its ranking performance.

However, it is important to note a well-known limitation of the Mean Average Precision metric: a model may achieve a high score by delivering high precision on a few queries while underperforming on the majority. To mitigate this, we incorporated into our heuristic the number of queries for which a model provides a more accurate ranking.

The Eq. (16) is a simple weighted sum borrowed by machine learning ensemble methods and models (Mohammed & Kora, 2023). Such fusion methods can be simplistic as the one implemented in our case or more sophisticated voting methods such as the Borda Count (de Borda, 1781) or the work of Markovits, Shtok, Kurland, and Carmel (2012).

Upon closer examination of the equation, $positives_{i,j}$ is computed by comparing the precision of $model_i$ with every other $model_j$ for each query. Thus, this component of the metric is precision-based. The second component is the model's Mean Average Precision (MAP), which is also a precision-oriented metric applied across document queries. Finally, the third component of the equation reflects the percentage of edges removed from the graph. Given these factors, we anticipate a strong correlation between Eq. (16) and precision-based metrics, especially when the $a + b$ parameters approaches 1.

In each of the subsequent sections, we will compare the models on the three different conditions presented in Section 3 (Eqs. (10), (12) and (14)). On each figure that the mean average precision is discussed, we will include a red horizontal line which will be the baseline of our comparison. That value will be derived from the best approach of the windowed model.

5.2. Evaluation and comparison with complete document graphs

In Fig. 2, the performance of the models is presented based on mean average precision and are shown on the left vertical axis, with the number of clusters generated by spectral clustering as the independent variable shown the horizontal axis represents the model and its corresponding number of clusters. On the right side of the vertical axis, the pruning percentage (green curve) for each model is displayed.

From Fig. 2, it is evident that the PGSB outperforms the CGSB in all cases, regardless of the number of clusters. Specifically, CGSB-110 achieves the highest performance with MAP of 0.24386, with the other CGSB models following closely behind. In contrast, some of the PGSB models perform worse than GSBW-7 (falling below the red line). Regarding pruning percentages, no specific conclusions can be drawn in relation to the number of clusters, other than that the pruning percentages range between [22.62, 34.07]

Fig. 3 presents the confusion matrix, which compares the performance of each query across different model pairs to identify the model with the greatest difference compared to the others. This matrix yields the first quantity of metric (16).

Finally, it is important to notice that the total of the answered questions between model pairs may not always equal 100, as questions with identical values were excluded.

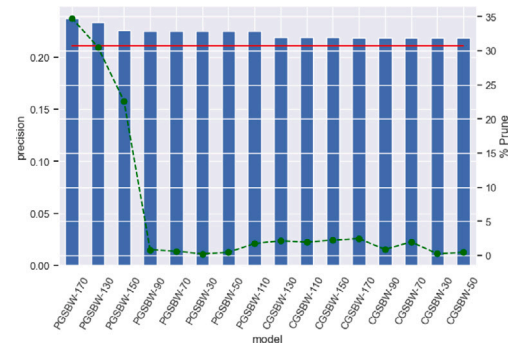


Fig. 2. Mean average precision & Pruning rate based on the number of clusters.

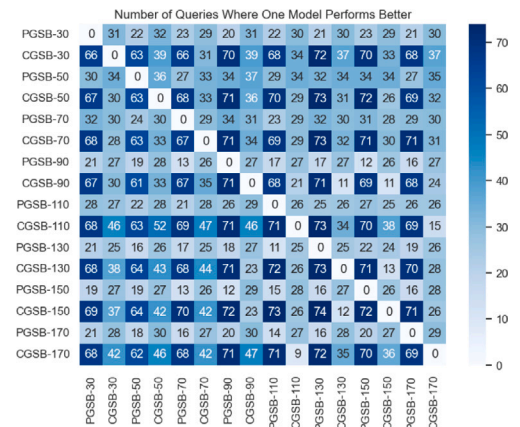


Fig. 3. Mean average precision & Pruning rate based on the number of clusters.

Fig. 4 displays the results of the experiments conducted with CGSB and PGSB models based on Eq. (12), along with the cluster and similarity values specified in Table 3. It is clear that CGSB models generally outperform PGSB models, except for the PGSB-90 and PGSB-110 models, when the similarity of node embeddings is 10% or higher. In all other comparisons, this is not the case, suggesting that this result might be considered noise. The MAP of the best models (CGSB-150%–10%, CGSB-150%–30%, CGSB-130%–50%, CGSB-150%–70%) across all similarity cases is 0.244. Due to the dominance of the 150 clusters in all similar cases, it can be concluded that this is the most suitable choice for this case.

Lastly, two additional observations are noteworthy. First, the relationship between the pruning rate and similarity is positively linear. This is logical, as increasing the threshold for similarity between two nodes makes it less likely for this condition to hold.

Secondly, as the similarity threshold between nodes increases, the performance of PGSB models deteriorates. This decline is due to how

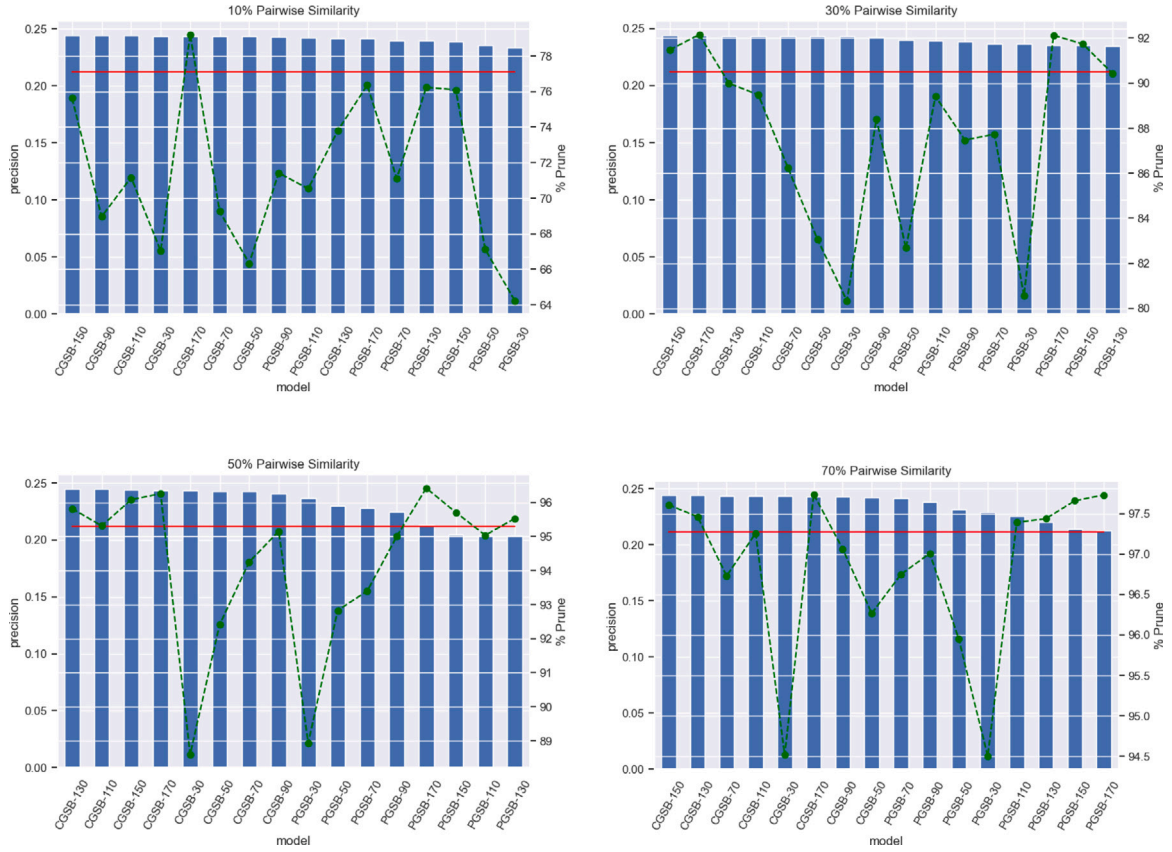


Fig. 4. Mean average precision & Pruning rate based on the number of clusters and node similarity on complete document graph.

term weights are computed in each model. As explained in a later section the NWk term weights in PGSB are calculated using Eq. (3), which is highly sensitive to the presence of neighboring nodes. At very high pruning rates (high similarity thresholds), crucial information needed for accurate weight calculation is lost due to the significant sparsity of the graph, leading to lower quality term representation based on NWk . In contrast, CGSB models demonstrate a much more stable performance with respect to the similarity threshold.

Fig. 5 presents the results of experiments conducted on CGSB and PGSB models using the cluster values and edge weight coefficients specified in Table. The average edge weight ($avg_{w_{out}}$) of the union graph is 8.2, and the pruning threshold is determined by multiplying this average by the respective coefficient. Once again, the CGSB models outperform the PGSB models, with the Mean Average Precision of the top models (CGSB-170-1.1, CGSB-170-1.2, CGSB-170-1.3, CGSB-110-1.4) being 0.244 for the first three cases and 0.243 for the last one. This suggests that 170 is the optimal choice for the number of clusters in CGSB. Finally, regarding the choice of the weight coefficient, the pruning rates are similar across the first three cases, but 1.2 offers a balanced intermediate solution.

5.3. Evaluation and comparison of windowed document graphs with constant size

In this section, the PGSBW and CGSBW models are examined with a fixed constant window size of 7. In the work of Kalogeropoulos et al. (2024) it is stated that the window size affects the precision of the models and for the Cystic fibrosis collection a size of 7 words is appropriate.

Fig. 6 shows the performance of the models in terms of Mean Average Precision, with the independent variable being the number of clusters. The red horizontal line represents $y = 0.211$, is used as a benchmark of comparison versus the simple model with window size

of 7 (GSB-7). Similarly, Fig. 7 presents the confusion matrix, which compares the performance of each question across model pairs.

From Fig. 6, it is evident that PGSBW outperforms CGSBW in all cases, regardless of the number of clusters. Specifically, PGSBW-170 achieves the highest Mean Average Precision of 0.236, followed by PGSBW-130 with an Mean Average Precision of 0.233. However, while all the new models outperform GSBW-7, they do not surpass the PGSB and CGSB models from the previous chapter. Additionally, note that the pruning rates in most cases (except for the first three) are close to zero. This occurs because spectral clustering results in a significant imbalance in the number of nodes per cluster, leading to fewer potential inter-cluster connections for pruning.

The confusion matrix (Fig. 7) further confirms that PGSBW not only achieves a higher Mean Average Precision but also answers a larger number of questions correctly, enhancing its reliability. Specifically, for the cluster range of [30 – 110], PGSBW answered nearly all questions better than CGSBW (>95%).

Fig. 8 shows the results of experiments conducted on PGSBW and CGSBW models based on similarity condition (Eq. (12)) using the cluster and similarity values. First, note that the PGSBW models consistently outperform the CGSBW models in all cases. Additionally, the behavior of the PGSBW models is nearly identical in terms of both Mean Average Precision and pruning rate for the similarity percentage pairs (10%, 30%) and (50%, 70%). The MAP of the top models (PGSBW-150-10%, PGSBW-150-30%, PGSBW-70-50%, PGSBW-90-70%) is 0.239 for the first two cases and 0.238 for the latter two.

Regarding the relationships between the hyperparameters, it is noted that the pruning rate has a positive linear relationship with similarity, which is a desirable outcome. Finally, an observation can be made that for the two highest similarity percentages (50%, 70%), the model with the best Mean Average Precision tends to have a lower number of clusters. This indicates that such high pruning rates are

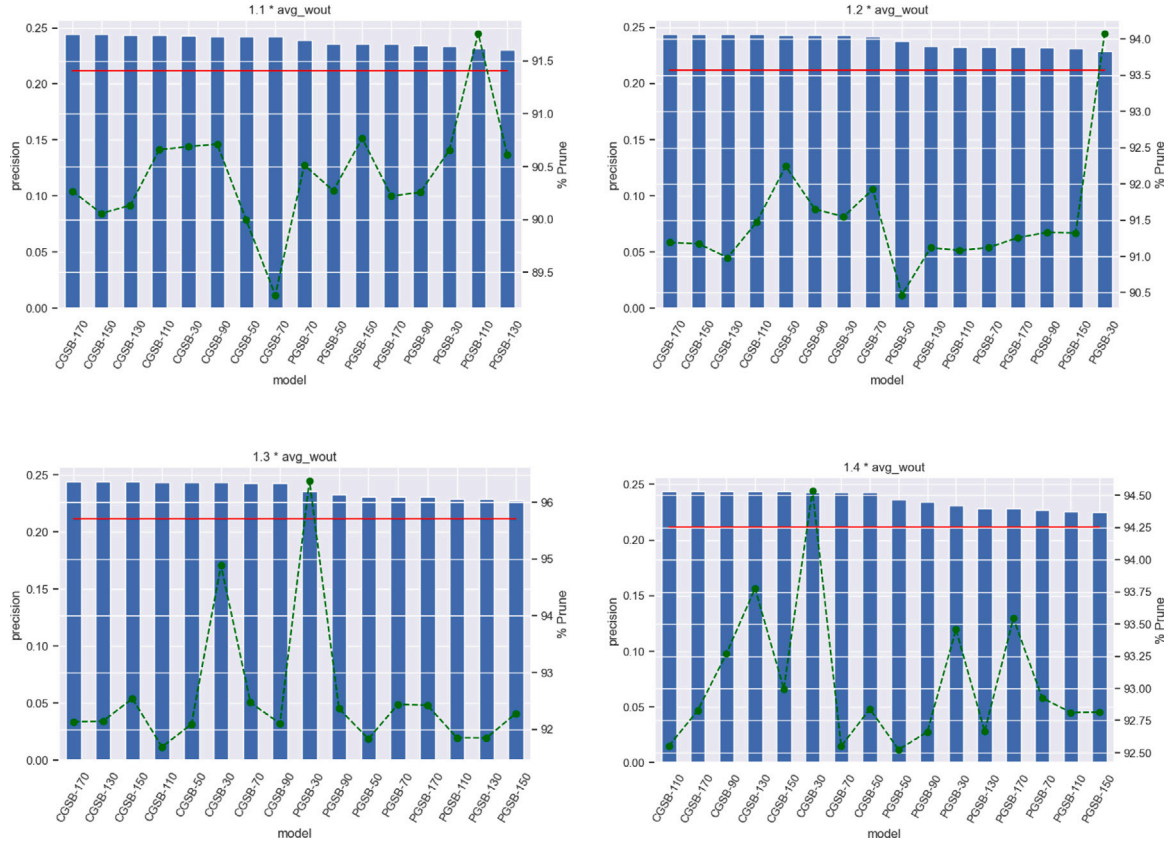


Fig. 5. Mean average precision and Prune rate based on number of clusters and average edge weights on complete document graphs.

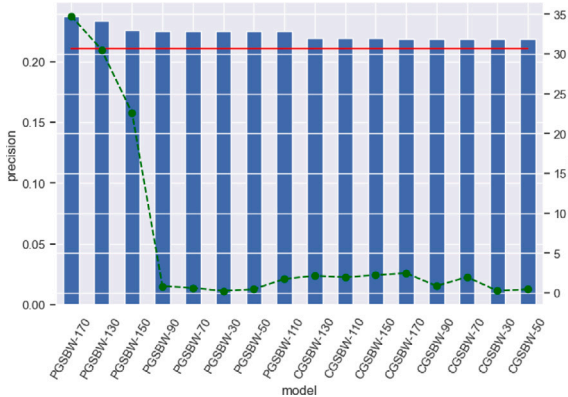


Fig. 6. Mean average precision & Pruning rate based on the number of clusters with constant window size of 7.

not necessary in models employing windows as a text segmentation strategy.

Fig. 9 shows the results of experiments conducted on the pruned and conceptualized models having their document graphs created implementing windows based cluster values and the average edge weight. The average edge weight (avg_wout) of the union graph for the Cystic Fibrosis collection in this case is 2.5, and the pruning threshold is determined by multiplying this average by the respective coefficient. In this experiment, the conceptualized models (CGSBW) models outperform the simple prune models (PGSBW) when the edge weight coefficient is greater than 1.1. The Mean Average Precision of the top models (PGSBW-170-1.1, CGSBW-170-1.2, CGSBW-170-1.3, CGSBW-30-1.4) is

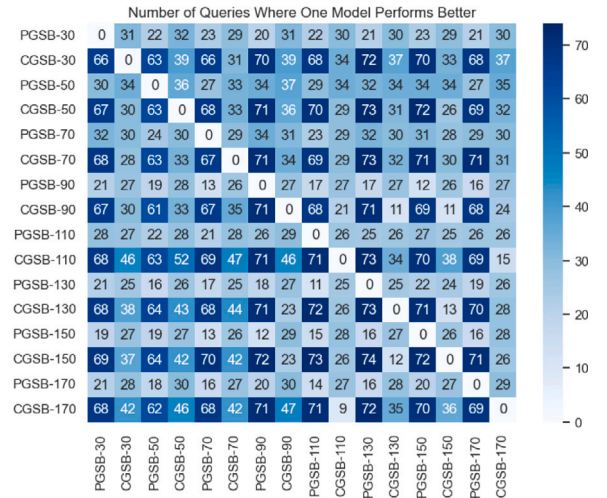


Fig. 7. Mean average precision & Pruning rate based on the number of clusters with constant window size of 7.

0.230, 0.233, 0.234, and 0.228, respectively. Therefore, it is reasonable to conclude that 170 is the optimal number of clusters in this approach.

5.3.1. Evaluation and comparison of document graphs with dynamic window size

Kalogeropoulos et al. (2024) highlighted the impact of window size on the precision of each model among multiple document collections. To address this, they proposed a simple yet effective method for calculating the window size based on the total document length, ensuring

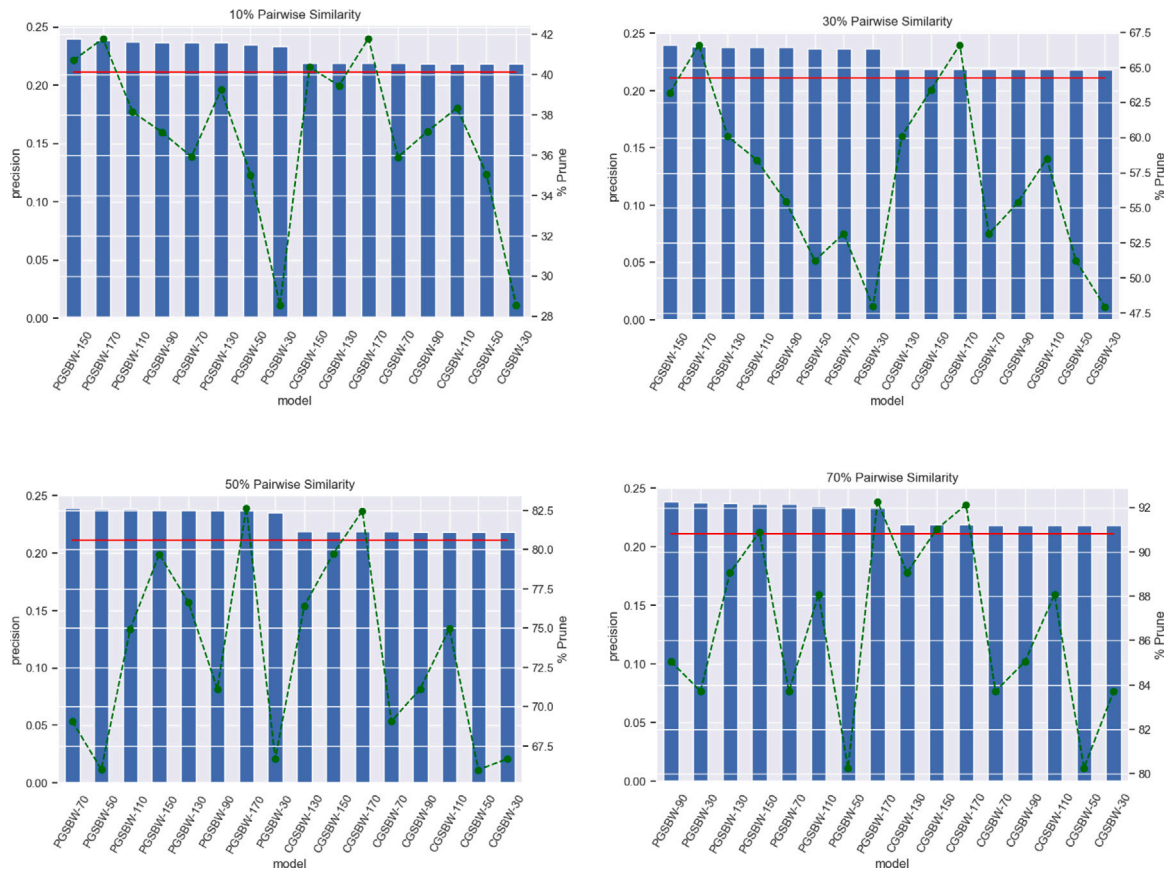


Fig. 8. Mean average precision and Prune rate based on number of clusters and node similarity on windowed document graphs.

Table 4

Model's performance implementing dynamic window sizes.

Type	Model	Number of clusters	Similarity/edge	Window %	MAP
Concept.	CGSBW-0.5-110-0.5	110	0.7	50%	0.217
Concept.	CGSBW-0.7-110-0.5	110	0.5	70%	0.217
Concept.	CGSBW-0.1-170-1.4	170	1.4	10%	0.228
Prun.	PGSBW-0.1-130-0.7	130	0.7	10%	0.240
Prun.	PGSBW-0.1-130-0.5	130	0.5	10%	0.238
Prun.	PGSBW-0.1-110-0.5	110	0.5	10%	0.243
Prun.	PGSBW-0.7-110-1.3	110	1.3	70%	0.235

that the graph-based extension consistently produces precise results across such collections. Table 4 shows the performance of that approach implementing the previously discussed methods.

When implementing the condition shown in Eq. (14), the conceptualized approach consistently outperforms the simple pruning model across all cluster sizes. However, while all the new models surpass the constant windowed sized model (GSBW-7), they do not outperform the original PGSB and CGSB models, which have their documents represented as complete graphs.

As the window size percentage increases, the MAP of the models correspondingly improves. This is anticipated, as they tend to gradually converge to the complete case as the window percentage approaches 100%. Additionally, there is a positive linear correlation between the window size percentage and the pruning rate, attributed to the model's inclusion of more correlated words, which results in an increased number of redundant edges. On small percentage values the results tends to be similar with the constant window method.

When taking into consideration the node similarity as denoted by Eq. (12) the conceptualized models under preform in comparison to the simple pruning models. The best Mean Average Precision is achieved by 110 clusters on a 10% window percentage with 50%

node similarity as shown in Table 4. The same outcome is observed when implementing the average edge weight method (Eq. (10)). The highest MAP is 0.228, which does not exceed that of the best model. It should be noted that while the pruned models improve when the window size percentage increases, the conceptualized do not in this case. However, increasing the window size percentage and similarity percentage enhances their performance. This indicates that the node embeddings become progressively more accurate and representative, based on the density of the initial union graph.

5.4. Evaluation and comparison of query expansion implementing spectral embeddings

In this subsection we will evaluate the implementation of spectral embeddings for query expansion in the most efficient model (CGSB-170-0.1) identified in the previous subsections. A comparison of the MAP is conducted using a difference plot to evaluate its quality. Fig. 10 illustrates the performance fluctuations between CGSB-170-0.1 and CGSB-170-0.1-QE, which is the query expanded model. The differences in their measurements are shown on the vertical axis. The horizontal baseline line ($y = 0$) helps identify where one model performs better

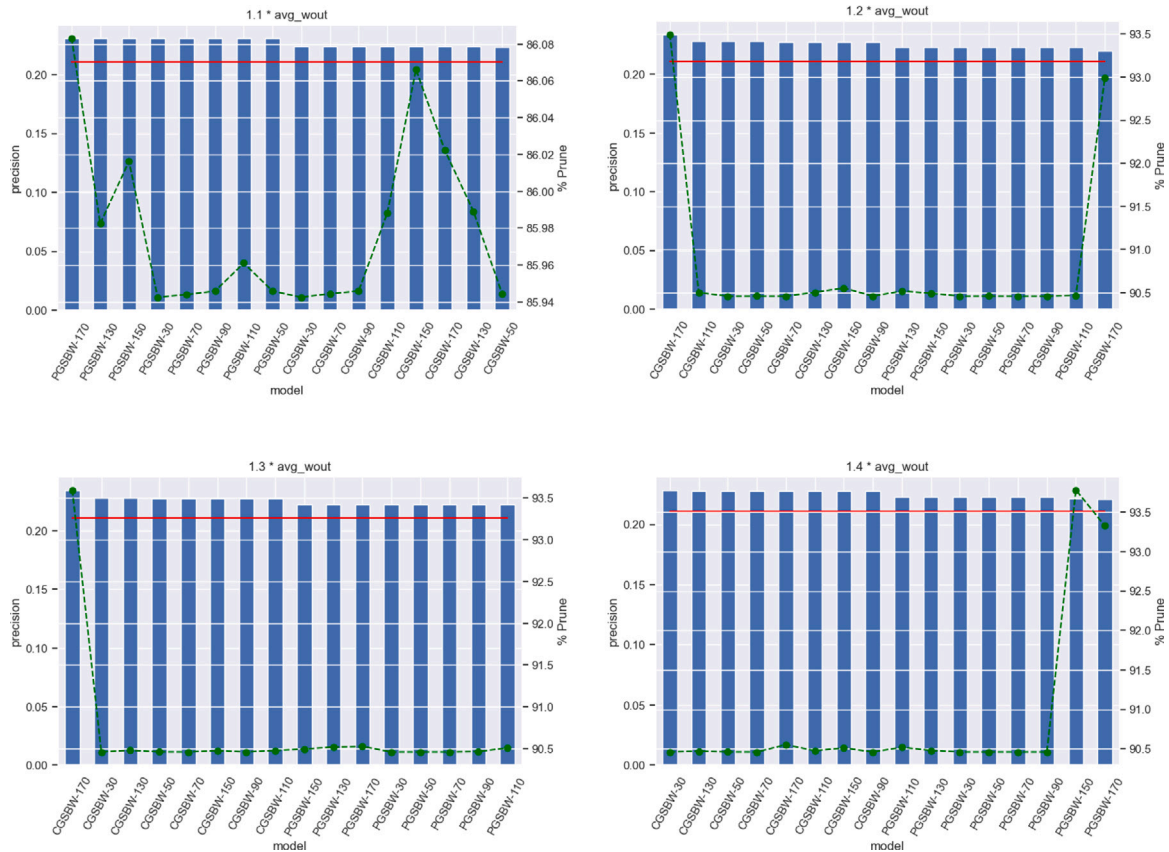


Fig. 9. Mean average precision and Prune rate based on number of clusters and average edge weights on windowed document graphs.



Fig. 10. Model comparison using query expansion.

than the other. Points above this line indicate superior performance by the CGSB-170-0.1-QE model, while points below the line indicate where CGSB-170-0.1 performs better.

It is observed that query expansion improved for 60 out of 100 queries, which represents a significant portion of the dataset. However, the magnitude of improvement for the enhanced queries is more than twice that of the deteriorating ones. Additionally, the overall Mean Average Precision shows an increase, rising from 0.243 to 0.257. This increase reflects a 5.76% improvement due to the application of the technique.

The distribution of performance differences between the CGSB-170-0.1 and CGSB-170-0.1-QE models exhibits a wide range of results. A notable frequency of 26 is concentrated around the narrow range $[-0.0092, 0.0073]$. Moreover, the distribution reveals more queries with positive improvements (as indicated by the number of bins with positive values) than negative declines. The most significant improvement lies within the range $[0.0073, 0.0239]$, while the most substantial drop

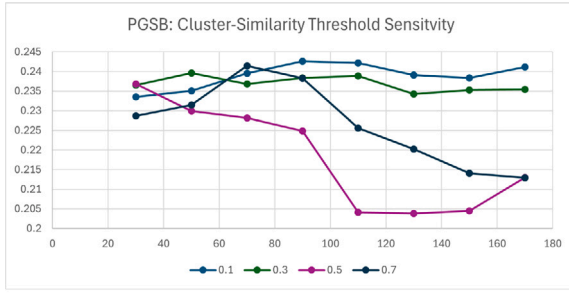
occurs between $[-0.0258, 0.0092]$. In many instances, the performance improvement is evident by the decline in frequency distribution leaning more towards the positive side. This skew towards positive differences suggests that the benefits of using query expansion in the CGSB-170-0.1-QE model generally outweigh its potential drawbacks, underscoring its effectiveness in this context.

An important topic for discussion concerns the number of terms selected for query expansion. In our approach, we added one query term from the embedding space for each non-stopword in the original query. Although we experimented with adding more related terms per query term, this led to a decline in model precision. The primary challenge in adding multiple terms lies in the increased complexity introduced to the underlying set-based model.

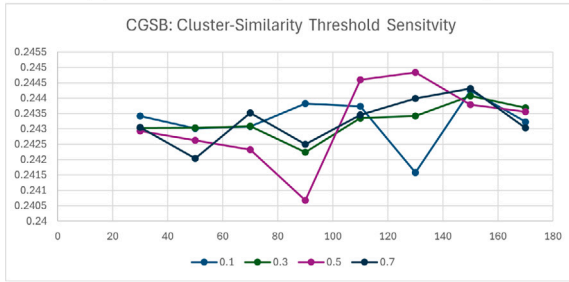
5.5. Choosing the best model

In the previous subsection, we applied query expansion to the model identified as the best performer. Table 5 presents the performance results of the experiments, depicting the precision of each model. Additionally, it provides a comparison with other state-of-the-art graph-based models, such as Graph of Words (Rousseau & Vazirgiannis, 2013) and models that implement contextual embeddings, such as ColBERT (Khattab & Zaharia, 2020).

On the number of cluster sensitivity from Figs. 11(a), 11(b), 12(a) and 12(b) we can observe that in each threshold case the concepts generated improve the conceptual model on the retrieval task judged by its Mean Average Precision, offering stable results in the range of $(0.24, 0.245)$. However, the simple pruned model seems to be easily affected by changes, thus reducing its sensitivity. From Fig. 11(a) It can be concluded that the performance of the pruned model is influenced more significantly by the similarity threshold than by the number of clusters. On the other hand, from Fig. 12(a) it can be easily observed

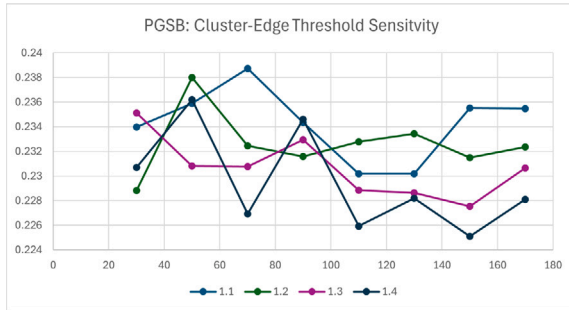


(a) PGSB: Similarity Threshold Sensitivity

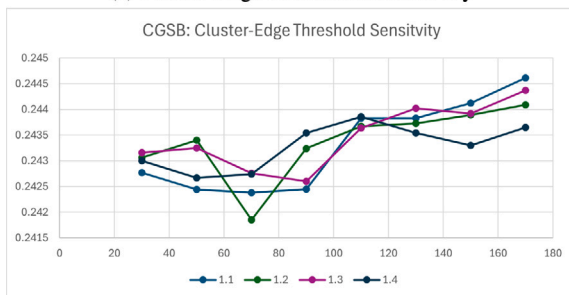


(b) CGSB: Similarity Threshold Sensitivity

Fig. 11. Cluster — Similarity sensitivity on PGSB and CGSB.



(a) PGSB: Edge Threshold Sensitivity



(b) CGSB: Edge Threshold Sensitivity

Fig. 12. Cluster — Edge threshold sensitivity on PGSB and CGSB.

that both the edge threshold and the number of clusters affect the models performance.

It is important to note that when the number of clusters is very large, the impact of intra-cluster pruning on the graph becomes minimal. As the cluster count approaches the number of unique terms in the collection, this effect diminishes further. Typically, the generated clusters contain only a single term, or occasionally a few terms in rare cases. In such instances, inter-cluster pruning becomes more relevant, removing edges based on a similarity or edge threshold. This approach yields

Table 5

Model evaluation implementing the mean average precision.

Model	MAP
CGSB-170-0.1	0.2432
CGSB-30-0.3	0.2430
CGSB-90-1.2	0.2432
CGSB-50-1.2	0.2434
CGSB-150-0.1	0.2442
CGSB-170-1.3	0.2443
PGSBW-0.1-130-0.7	0.2403
CGSB-170-1.2	0.2440
CGSB-130-0.7	0.2439
CGSB-170-1.4	0.2436
CGSB-170-0.1-QE	0.257
Set Based Model	0.1618
GSB	0.1861
W-GSB	0.2110
BM25	0.2226
W-GSB+colBERT	0.2311
colBERT	0.2291
GoW	0.1173

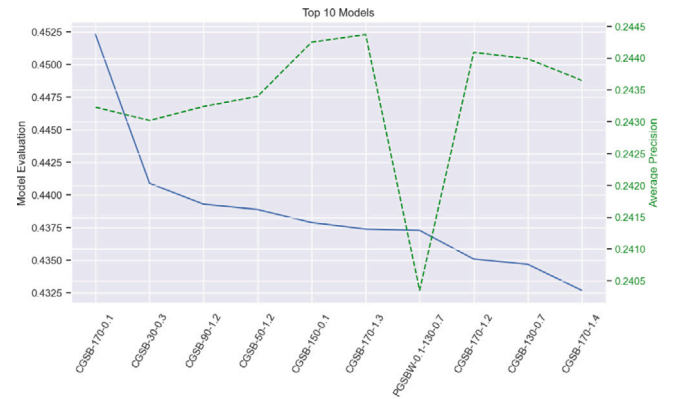


Fig. 13. Model evaluation implementing the heuristic function.

Mean Average Precision results that fall between those of the simple GSB and W-GSB models. Specifically, the model behaves like a pruned by edges GSB model (Kalogeropoulos et al., 2020).

Therefore, in order to select parametrize each model correctly and finally to select the optimal, we employ the evaluation metric introduced in Eq. (16). In the equation used, we chose α to be 0.5, β to be 0.4 and finally γ 0.1. Those values represent the importance that we deem appropriate for each variable. It is important to note that the precision based parts of the equation will be the main criterion on the evaluation as the $\alpha + \gamma$ is close to 1.

We pick the best model of each category, thus a total of 69 models were chosen for comparison, with 27 models selected from the first category (PGSB, CGSB), another 27 from the second category (PGSBW-7, CGSBW-7), and 15 models from the third category (PGSBW_{per}, CGSBW_{per}). Fig. 13 shows the ten best candidates.

An interesting observation is that the model that is ranked first does not achieve the highest precision. This happens because our metric takes also into account the size of the respective union graph, and the number of queries for which a model provides a more accurate ranking. Hence the model that appears to be the best CGSB-170-0.1 achieves the better compromise among these three performance indicators and hence it can be considered to be the best choice.

Last but not least, we addressed the number of iterations required for Spectral Clustering to converge in the CGSB model, a consideration that can be generalized to all proposed methods. The algorithm used to assign each node to a cluster is k-means. In Fig. 14 the iterations required for convergence are shown for each parameter. On average, the model converges in 41 iterations. Therefore, the clustering does not

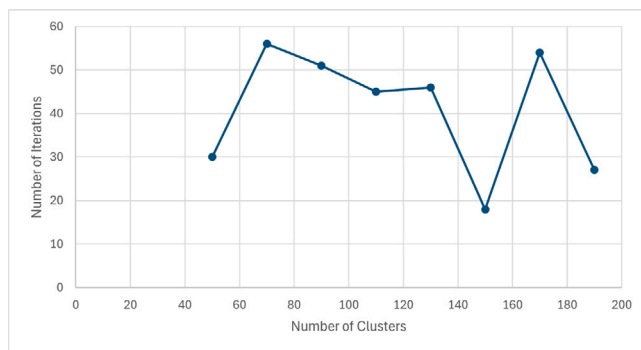


Fig. 14. Clustering convergence.

hinder the model with long convergence iterations which will deem the model nonapplicable

6. Conclusions and future work

In this paper, we introduced and applied several graph pruning methods, along with a conceptual ranking scheme that leverages spectral clustering. The resulting models effectively capture both semantic and syntactic information relying on a much smaller collection-wide union graph. Notably, the inclusion of conceptual elements has significantly enhanced the performance of the ranking model, surpassing earlier versions and even some state-of-the-art models.

Our evaluation encompassed not only the ranking performance but also the required size of the union graph to ensure a thorough assessment of the proposed models and methods. To facilitate this comprehensive evaluation, we employed a heuristic linear combination as the evaluation function.

Our approach also generates node embeddings as a byproduct to the spectral clustering, which can be applied to various machine learning tasks. Incorporating relevant terms through query expansion using these embeddings aids the ranking scheme, resulting in a highly accurate model. This suggests that the generated embedding vectors are capable of capturing the relationships among terms and should therefore be effective in addressing related challenges.

The use of intermediate methods such as pruning or conceptualization through spectral clustering results in increased time complexity. This introduces a primary limitation of our approach: the necessity of applying spectral clustering to each change in the data collection to ensure the accuracy of node embeddings. However, it can be noted that if the data volume is sufficiently large, the need to generate new embeddings with each change is minimized. Consequently, a future research direction should involve estimating the data volume required to produce robust embeddings and determining the frequency or number of changes necessary for the model to be reapplied effectively.

Furthermore, the conceptualized method demonstrated robust performance across varying numbers of clusters in most cases, indicating that this parameter does not require tuning through methods such as silhouette score, elbow, or eigen gap, which would add complexity to the model. However, the performance of the simpler pruned model remains sensitive to the number of clusters.

Therefore, future research should explore optimizing the number of clusters by using precision as the loss function and applying deep learning techniques. This approach could be implemented across multiple collections, incorporating a diverse range of documents and queries to enhance the model's generalization and effectiveness.

Finally, another area of future research would be to explore the role of nodes or links that may not be well-connected within a cluster but still have a significant impact on the topic of interest. The model can handle separated graphs, however, If such critical nodes or links are

pruned during the graph construction or clustering process, it could lead to unintended consequences. Investigating these cases from an operational perspective could provide valuable insights and help refine the models to better account for the influence of these key elements in the graph structure.

CRedit authorship contribution statement

Nikitas-Rigas Kalogeropoulos: Conceptualization, Methodology/Study design, Software, Validation, Formal analysis, Investigation, Resources, Writing – original draft, Writing – review & editing, Visualization. **George Kontogiannis:** Conceptualization, Methodology/Study design, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Christos Makris:** Conceptualization, Methodology/Study design, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We have shared our data and source code on a public repository for reproducibility.

References

- Belkin, M., & Niyogi, P. (2003). Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6), 1373–1396. <http://dx.doi.org/10.1162/089976603321780317>.
- Bonald, T. (2019). *Lecture notes, Spectral graph embedding*. Institut Polytechnique de Paris.
- Borlea, I.-D., Precup, R.-E., & Borlea, A. (2022). Improvement of K-means cluster quality by post processing resulted clusters. *Procedia Computer Science*, 199, 63–70. <http://dx.doi.org/10.1016/j.procs.2022.01.009>.
- Borlea, I.-D., Precup, R.-E., Dragan, F., & Borlea, A.-B. (2017). Centroid update approach to k-means clustering. *Advances in Electrical and Computer Engineering*, 17(4), 3–10. <http://dx.doi.org/10.4316/AECE.2017.04001>.
- Camacho-Collados, J., & Pilehvar, M. T. (2018). From word to sense embeddings: A survey on vector representations of meaning. *Journal of Artificial Intelligence Research*, 63, 743–788.
- Dai, Z., & Callan, J. (2019). Deeper text understanding for IR with contextual neural language modeling. In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval* (pp. 985–988).
- de Borda, J. C. (1781). *Mémoire sur les élections au scrutin*. Histoire de l'Académie Royale des Sciences.
- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 391–407. [http://dx.doi.org/10.1002/\(SICI\)1097-4571\(199009\)41:6<391::AID-AS11>3.0.CO;2-9](http://dx.doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-AS11>3.0.CO;2-9).
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805).
- Diaz, F., Mitra, B., & Craswell, N. (2016). Query expansion with locally-trained word embeddings. arXiv preprint [arXiv:1605.07891](https://arxiv.org/abs/1605.07891).
- Gálvez, J., Cuevas, E., Becerra, H., et al. (2020). A hybrid optimization approach based on clustering and chaotic sequences. *International Journal of Machine Learning and Cybernetics*, 11, 359–401. <http://dx.doi.org/10.1007/s13042-019-00979-6>.
- Grover, A., & Leskovec, J. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 855–864).
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30.
- Kalogeropoulos, N.-R., Doukas, I., Makris, C., & Kanavos, A. (2020). A graph-based extension for the set-based model implementing algorithms based on important nodes. In I. Maglogiannis, L. Iliadis, & E. Pimenidis (Eds.), *Artificial intelligence applications and innovations. AIAI 2020 IFIP WG 12.5 international workshops* (pp. 143–154). Cham: Springer International Publishing.

- Kalogeropoulos, N. R., Skamnelos, N., Makris, C., et al. (2024). On the graph-based extension of the set-based model: A new approach on graph representation and model ensemble. *Research Square*, <http://dx.doi.org/10.21203/rs.3.rs-3801152/v1>, PREPRINT (Version 1) available at Research Square.
- Khattab, O., & Zaharia, M. (2020). Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (pp. 39–48).
- Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv preprint [arXiv:1609.02907](https://arxiv.org/abs/1609.02907).
- Markovits, G., Shtok, A., Kurland, O., & Carmel, D. (2012). Predicting query performance for fusion-based retrieval. In *Proceedings of the 21st ACM international conference on information and knowledge management* (pp. 813–822).
- Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing order into text. In *Proceedings of the 2004 conference on empirical methods in natural language processing* (pp. 404–411). Barcelona, Spain: Association for Computational Linguistics, URL: <https://aclanthology.org/W04-3252>.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint [arXiv:1301.3781](https://arxiv.org/abs/1301.3781).
- Mohammed, A., & Kora, R. (2023). A comprehensive review on ensemble deep learning: Opportunities and challenges. *Journal of King Saud University - Computer and Information Sciences*, 35(2), 757–774. <http://dx.doi.org/10.1016/j.jksuci.2023.01.014>, URL: <https://www.sciencedirect.com/science/article/pii/S1319157823000228>.
- Narayanan, A., Chandramohan, M., Venkatesan, R., Chen, L., Liu, Y., & Jaiswal, S. (2017). Graph2vec: Learning distributed representations of graphs. arXiv preprint [arXiv:1707.05005](https://arxiv.org/abs/1707.05005).
- Nogueira, R., Yang, W., Cho, K., & Lin, J. (2019). Multi-stage document ranking with BERT. arXiv preprint [arXiv:1910.14424](https://arxiv.org/abs/1910.14424).
- Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The pagerank citation ranking: Bringing order to the web: Technical report 1999-66*, Stanford InfoLab, URL: <http://ilpubs.stanford.edu:8090/422/>, Previous number=SIDL-WP-1999-0120.
- Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global vectors for word representation. In A. Moschitti, B. Pang, & W. Daelemans (Eds.), *Proceedings of the 2014 conference on empirical methods in natural language processing* (pp. 1532–1543). Doha, Qatar: Association for Computational Linguistics, <http://dx.doi.org/10.3115/v1/D14-1162>, URL: <https://aclanthology.org/D14-1162>.
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 701–710).
- Possas, B., Ziviani, N., Meira, W., Jr., & Ribeiro-Neto, B. (2002). Set-based model: A new approach for information retrieval. In *Proceedings of the 25th annual international ACM SIGIR conference on research and development in information retrieval* (pp. 230–237). <http://dx.doi.org/10.1145/564376.564417>, URL: <http://doi.acm.org/10.1145/564376.564417>.
- Robertson, S. E., & Sparck Jones, K. (1977). Relevance weighting of search terms. *Journal of the American Society for Information Science*.
- Robertson, S. E., Zaragoza, H., & Taylor, M. (2004). Simple BM25 extension to multiple weighted fields. In *Proceedings of the 2004 ACM CIKM international conference on information and knowledge management* (pp. 42–49). ACM.
- Rousseau, F., & Vazirgiannis, M. (2013). Graph-of-word and TW-IDF: New approach to ad hoc IR. In *Proceedings of the 22nd ACM international conference on information and knowledge management* (pp. 59–68). ACM, <http://dx.doi.org/10.1145/2505515.2505671>.
- Rousseau, F., & Vazirgiannis, M. (2015). Main core retention on graph-of-words for single-document keyword extraction. In *Advances in information retrieval* (pp. 382–393). Springer International Publishing.
- Roy, D., Paul, D., Mitra, M., & Garain, U. (2016). Using word embeddings for automatic query expansion. arXiv preprint [arXiv:1606.07608](https://arxiv.org/abs/1606.07608).
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint [arXiv:1910.01108](https://arxiv.org/abs/1910.01108).
- Seidman, S. B. (1983). Network structure and minimum degree. *Social Networks*, 5(3), 269–287. [http://dx.doi.org/10.1016/0378-8733\(83\)90028-X](http://dx.doi.org/10.1016/0378-8733(83)90028-X).
- Selva Birunda, S., & Kanniga Devi, R. (2021). A review on word embedding techniques for text classification. In J. S. Raj, A. M. Iliyasa, R. Bestak, & Z. A. Baig (Eds.), *Innovative data communication technologies and application* (pp. 267–281). Singapore: Springer Singapore.
- Shaw, M., Wood, B., Wood, B., & Tibbo, R. (1991). *The cystic fibrosis database: Content and research opportunities*.
- Sonawane, S., & Kulkarni, P. (2014). Graph based representation and analysis of text document: A survey of techniques. *International Journal of Computer Applications*, 96, 1–8. <http://dx.doi.org/10.5120/16899-6972>.
- Sun, Y., Han, J., Yan, X., Yu, P. S., & Wu, T. (2011). Paths: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, 4(11), 992–1003.
- Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., & Mei, Q. (2015). Line: Large-scale information network embedding. In *Proceedings of the 24th international conference on world wide web*. URL: <https://api.semanticscholar.org/CorpusID:8399404>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv preprint [arXiv:1710.10903](https://arxiv.org/abs/1710.10903).
- von Luxburg, U. (2007). A tutorial on spectral clustering. *Statistics and Computing*, 17(4), 395–416, URL: <https://www.springer.com/article/10.1007/s11222-007-9033-z>, Max Planck Institute for Biological Cybernetics, Spemannstr. 38, 72076 Tübingen, Germany.
- Wang, J., Huang, J. X., Tu, X., Wang, J., Huang, A. J., Laskar, M. T. R., et al. (2024). Utilizing BERT for information retrieval: Survey, applications, resources, and challenges. *ACM Computing Surveys*, 56(7), <http://dx.doi.org/10.1145/3648471>.
- Zhou, F., Mahler, S., & Toivonen, H. (2012). Simplification of networks by edge pruning. In *Bisociative knowledge discovery: An introduction to concept, algorithms, tools, and applications* (pp. 179–198). Springer Berlin Heidelberg, http://dx.doi.org/10.1007/978-3-642-31830-6_13.

Nikitas-Rigas Kalogeropoulos born on 18th November 1991 and is a PhD candidate at the University of Patras, focusing on "Text Representation and Modeling with Applications on Information Retrieval and Data Mining." He earned his Master's degree in Data-Driven Computing and Decision Making from the University of Patras, where his thesis, supervised by As. Prof. Christos Makris, was on "Text Topic Modeling and Information Retrieval in Distributed Systems." Kalogeropoulos also holds a Diploma in Computer Engineering and Informatics from the University of Patras, with a thesis on "Text Representation Implementing Graphs," also supervised by Associate Professor Makris. He has extensive teaching experience at the University of Patras, serving as a teaching assistant for courses such as Introduction to Bioinformatics, Information Retrieval, Data Mining and Machine Learning, Databases, Introduction to Computer Science, and Object-Oriented Programming. He has also been involved in research projects and published research findings in several peer-reviewed conferences and journals.

George Kontogiannis, born on July 5th, 1999, in Nafpaktos, Greece, is a Ph.D. candidate at the University of Patras, specializing in artificial intelligence and digital signal processing. He entered the School of Engineering at the University of Patras in 2018 and earned his Integrated Master's degree in Computer Engineering in 2023. His thesis, supervised by Prof. Christos Makris, focused on improving information retrieval models using machine learning techniques applied to graph-based systems. Kontogiannis has been involved in multiple European-funded research projects and industry collaborations, contributing to the development of AI models for time series analysis, encompassing audio, kinesiological data, and vital signs. Additionally, he is an active contributor to the open-source community, particularly in the development of CrossAI-TS, a Python library designed for time series analysis in audio, voice, and other related AI tasks.

Christos Makris entered the Department of Computer Engineering and Informatics, School of Engineering, University of Patras in September 1988 and he received his Diploma, as a Computer Engineer, in December 1993. In November 1997 he received his Ph.D. in Computer Science. The title of his dissertation was "Elementary Data Structures and Their Application in Computational Geometry and Information Retrieval" and his Advisor was Professor Athanasios Tsakalidis. From November 1997 to May 1999 he has served his military service. From May 1999 till October 2002 he cooperated with the Research Academic Computer Technology Institute, From August 2003 to October 2004 he was an Assistant Professor in the Department of Applied Informatics in Management & Finance, Technological Educational Institute of Mesolonghi, while from October 2004 to September 2017 he was Assistant Professor in the Department of Computer Engineering and Informatics, University of Patras. From September 2017 till today, he is an Associate Professor in the same Department. From October 2005 till today, he is (on an annual basis) a member of the collaborating teaching staff of the Hellenic Open University.